

UNIT 4

Contents:

The Basics of NumPy Arrays - The Basics of NumPy Arrays - Aggregations: Min, Max, and Everything in Between - Computation on Arrays: Broadcasting - Fancy Indexing

Data Manipulation with Pandas - Introducing Pandas Objects - Data Indexing and Selection - Handling Missing Data - Combining Datasets: Concat and Append – Aggregation and Grouping. Visualization with Matplotlib - Importing matplotlib - Simple Line Plots - Simple Scatter

Plots - Histograms, Binnings, and Density – case study: Effect of Holidays on US Births

Introduction to Numpy

• **Python Lists vs NumPy Arrays – What's the Difference Python Lists vs NumPy Arrays ?**

- why use NumPy arrays when we already have Python lists? After all, these Python lists act as an array that can store elements of various types.
- This is a perfectly valid question and the answer to this is hidden in the way Python stores an object in memory.
- A Python object is actually a pointer to a memory location that stores all the details about the object, like bytes and the value. Although this extra information is what makes Python a dynamically typed language, it also comes at a cost which becomes apparent when storing a large collection of objects, like in an array.
- Python lists are essentially an array of pointers, each pointing to a location that contains the information related to the element.

- This adds a lot of overhead in terms of memory and computation. And most of this information is rendered redundant when all the objects stored in the list are of the same type!
- **To overcome this problem, we use NumPy arrays that contain only homogeneous elements**, i.e. elements having the same data type.
- This makes it more efficient at storing and manipulating the array.
- This difference becomes apparent when the array has a large number of elements, say thousands or millions.
- **Also, with NumPy arrays, you can perform element-wise operations, something which is not possible using Python lists!**
- This is the reason why NumPy arrays are preferred over Python lists when performing mathematical operations on a large amount of data.

What is a pip?

- Pip is a standard package management system used to install and manage the software packages written in python.
- We can use pip to install additional packages that are not available in the Python standard library
- **What are the various commands of pip?**
- Below are the various commands of pip to be run in the command prompt/terminal
- List all the packages-> pip list
- Install package -> pip install package Name; an example is: **pip install Numpy**
- Uninstall package -> pip uninstall package Name
- Search a package -> pip search package Name

What is NumPy and Why NumPy is an important library to work with data?

- NumPy is a Python library widely used to handle arrays with arrays.
- Numpy can handle oversized, multi-dimensional arrays and matrices, along with a large collection of mathematical operations to operate on these arrays.
- It stands for numerical python. NumPy can provide an array object that is **50 times faster** than traditional Python lists.
- An array occupies less memory and is extremely convenient to use as compared to python lists.

- Additionally, it has a mechanism for specifying the data types. NumPy can operate on individual elements in the array without using loops and list comprehensions.
- Now, before diving deep into the concept of NumPy arrays, it's important to note that Python lists can very well perform all the actions that NumPy arrays perform.
- It is simply the fact that NumPy arrays are faster and more convenient than lists when it comes extensive computations, which make them extremely useful, especially when you are working with a large amount of data

Installing NumPy in your System

\$ pip install numpy

Once Numpy is successfully installed in your system

import numpy as np

where np is an alias used for NumPy so that the NumPy package can be referred to as np instead of numpy.

two ways to create NumPy arrays

1.By converting the existing lists or tuples to arrays using np.array

2.By initializing fixed-length arrays using the Numpy functions.

#Creating a 1D numpy array

```
arr = np.array([1,2,3,4,5])
```

#Creating a 2D array

```
arr = np.array( [ [1,2,3],  
                 [4,5,6] ] )
```

Creating array of ones

`np.ones(5) #1D`

`np.ones( (3,5) ) #2D`

Change the default dtype of an array

`np.ones( 5) ,dtype = np.int) # creating array of integers`

`np.ones( 5) ,dtype = np.float ) # creating array of floats`

Creating array of random numbers

`np.random.random( [3,4] )`

#creating an array of numbers 1 to 100 with step of 5

`np.arange(1,100,5)`

#Creating an array of length 25 between 1 to 10

`np.linspace(1,10,25) # It equally divides 1 to 10 in 25 parts`

#Creating an array of length 25 between 1 to 10

`np.linspace(1,10,25)` # It equally divides 1 to 100 in 25 parts

To create an array of repeating sequence

`np.tile( [1,2,3] , 3 )` # The op will look like [1,2,3,1,2,3,1,2,3]

Creating an array of identity matrix

`np.eye(3, dtype = int )` # it will create an 3x3 identity matrix

#create a matrix of random integer between a particular range

`np.random.randint(0, 10, (3,3))` #3x3 array of random integers ranging from 0 to 9

creating uninitialized array of specific shape and dtype

`np.empty( [3,3] , dtype = int )`

#create an array of specific shape filled with zeroes

```
np.zeros( [3,3] , dtype = int )
```

Checking shape of an array

```
arr = np.array( [1,2,3] )
```

```
arr.shape
```

#checking dimensions of an array

```
arr.ndim
```

#checking shape of an array

```
arr.size
```

#checking the dtype of an array

```
np.dtype
```

Reshaping the NumPy arrays

```
arr = np.array( [1,2,3,4,5,6,7,8,9] )
```

```
arr.reshape( [3,3] ) # convering above array in 3x3 matrix
```

#flatenning the array

```
y = np.array( [[1,1], [1,1] ] )
```

```
y.flatten() # output : [1,1,1,1]
```

Array Indexing

```
array_1d[2] # Third element
```

```
array_1d[2:] # Third element onwards
```

```
array_1d[:3] # First three elements
```

```
array_1d[2:7] # Third to seventh elements
```

```
array_1d[0::2] # Subset starting 0 at increment of 2
```

```
print(arr[0]) #it will print first row
```

```
print(arr[1]) #it will print second row
```

```
print(arr[0][1]) #it will print second element of first row
```

```
arr=np.array([[81,19,46,74,94],[69,79,26,7,29],[21,45,12,80,72] ])
```

```
print(arr[2][1]) #it will print second element of third
```

Array Slicing

```
arr=np.array([0,1,2,3,4,5,6,7,8,9,10])
```

```
print(arr[:5]) #by default its startindex start from [0 1 2 3 4]
```

```
print(arr[5:]) #by default its stopindex go till last [ 5 6 7 8 9 10]
```

```
print(arr[4:10:2]) #output : [4 6 8]
```

```
print(arr[:,]) #by default it startindex is 0 stopin [0 1 2 3 4 5 6 7 8 9 10]
```

```
arr=np.array([[1,2,3,4,5,6],[7,8,9,10,11,12],[13,14,15,16,17,18]])
```

```
print(arr[0,1:5]) #it select first row and element from 1
```

```
#output : [2 3 4 5]
```

```
print(arr[1:4,2:4]) #it select row from first to third and
```

```
# output : [[ 9 10] [15 16]]
```

NumPy Aggregation Functions:

- `np.sum()`: Calculates the sum of all elements in an array or along a specified axis.
- `np.mean()`: Computes the arithmetic mean (average) of elements.
- `np.max()`: Finds the maximum value.
- `np.min()`: Finds the minimum value.
- `np.prod()`: Calculates the product of all elements.
- `np.std()`: Computes the standard deviation.
- `np.var()`: Computes the variance.
- `np.median()`: Calculates the median of the elements.
- `np.percentile()`: Computes the q-th percentile(s) of the data along the specified axis.
- `np.argmin()`: Returns the index of the minimum value.
- `np.argmax()`: Returns the index of the maximum value.

Concatenation

Concatenate() function can join the sequence of arrays that we provide, along with the axis specified. If the axis is not explicitly given, it is taken as 0

#Concatenating 1D array

```
arr1=np.array([1,2,3])
```

```
arr2=np.array([4,5,6])
```

```
arr=np.concatenate((arr1,arr2),axis=0)
```

```
print(arr)
```

#output : [1 2 3 4 5 6]

#Concatenating @d array along axis 0

```
arr1=np.array([[1,1,1],[1,1,1]])
```

```
arr2=np.array([[9,9,9],[9,9,9]])
```

```
arr=np.concatenate((arr1,arr2),axis=0)
```

```
print(arr)
```

#output : [[1 1 1]

[1 1 1]

[9 9 9]

[9 9 9]]

Joining array using Stack function

Stacking is the same as concatenation, the only difference is that stacking is done along a new axis.

np.hstack() :

stacking two 1D arrays horizontally

```
arr1=np.array([0,0])
```

```
arr2=np.array([1,1])
```

```
arr=np.hstack((arr1,arr2))
```

Stacking two 2D arrays horizontally

```
arr1=np.array([[0,0],[0,0]])
```

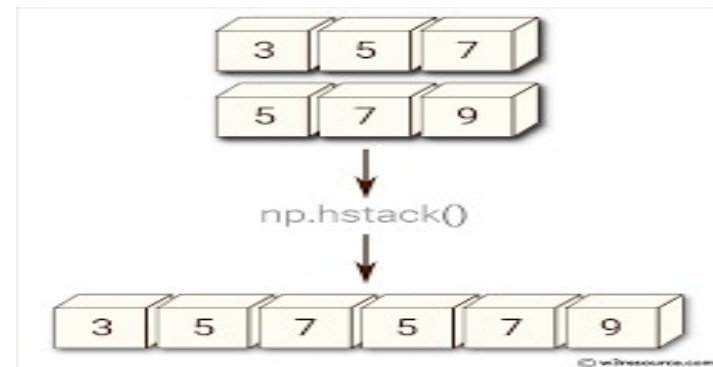
```
arr2=np.array([[1,1],[1,1]])
```

```
arr=np.hstack((arr1,arr2))
```

```
print(arr)
```

```
output : [[0 0 1 1]
```

```
[0 0 1 1]]
```



np.vstack() :

#Stacking 1D arrays vertically

```
arr1=np.array([0,0])
```

```
arr2=np.array([1,1])
```

```
arr=np.vstack((arr1,arr2))
```

```
print(arr)
```

```
#output :[[0 0]
```

```
         [1 1]]
```

Stacking 2D arrays vertically

```
arr1=np.array([[0,0],[0,0]])
```

```
arr2=np.array([[1,1],[1,1]])
```

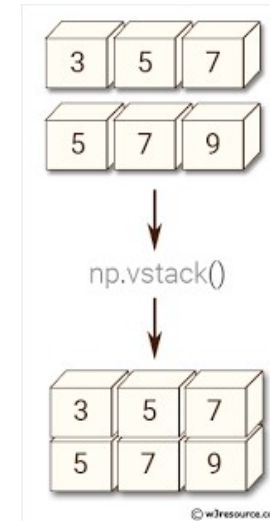
```
arr=np.vstack((arr1,arr2))
```

```
print(arr)
```

```
#output : [[0 0] [0 0]
```

```
          [1 1]
```

```
          [1 1]]
```



Sort the arrays

sorting means simply arranging el

```
arr=np.array([3,2,0,1])
```

```
print(np.sort(arr))
```

#output : [0 1 2 3]ements in an ordered sequence

Operations on Arrays

Array Addition:

array_1 + array_2

```
np.array([0,1]) + np.array([2,3])
```

Array Subtraction

- `array_1 - array_2`

`np.array([0,1]) + np.array([2,3])`

Raising by a power :

`Some_array ** n` , where n is any real number

`np.array([0,1])**2` #squaring

`np.array([1,2]) ** 1.2` #raising power by 1.2

```
a = np.array([1,4,2,5,3,6,8,7,9])
```

```
np.sort(a, kind='quicksort')
```

Output?

Sort the array along any axis you desire

```
a = np.array([[5,6,7,4],
```

```
               [9,2,3,7]])# sort along the column
```

```
print('Sort along column :','\n',np.sort(a, kind='mergesort',axis=1))
```

```
# sort along the row
```

```
print('Sort along row :','\n',np.sort(a, kind='mergesort',axis=0))
```

Output:

- **Broadcasting in NumPy arrays – A class apart!**
- Broadcasting is one of the best features of ndarrays. It lets you perform arithmetics operations between ndarrays of different sizes or between an ndarray and a simple number!
- The term **broadcasting** refers to the ability of NumPy to treat arrays of different shapes for performing operations.
- If the dimensions of two arrays are dissimilar, element-to-element operations are not possible. However, operations on arrays of non-similar shapes is still possible in NumPy, because of the broadcasting capability.
- The smaller array is **broadcast** to the size of the larger array so that they have compatible shapes.

- broadcasting describes how numpy treats arrays with different shapes during arithmetic operations.
- Aligned dimensions are equal.
- One of the aligned dimensions is 1

Broadcasting is a Tensor operation. Helpful in Neural Network (ML, AI)

Broadcasting a 1D Array to a 2D Array:

This example shows how a 1D array `a1` is added to a 2D array `a2`.

NumPy automatically expands the 1D array along the rows of the 2D array to perform element-wise addition.

```
import numpy as np
a1 = np.array([2, 4, 6])
a2 = np.array([[1, 3, 5], [7, 9, 11]])
res = a1 + a2
print(res)
```

Output:

```
[[ 3  7 11] [ 9 13 17]]
```

- It allows us to perform addition on arrays of different shapes without needing to reshape them.

Mean, Median and Standard deviation

```
a = np.arange(5,15,2)
print('Mean :',np.mean(a))
print('Standard deviation :',np.std(a))
print('Median :',np.median(a))
```

Output:?

Min-Max values and their indexes

```
a = np.array([[1,6],
[4,3]])
# minimum along a column
print('Min :',np.min(a,axis=0))
# maximum along a row
print('Max :',np.max(a,axis=1))
```

we can also easily determine the index of the minimum or maximum value in the ndarray along a particular axis using the **argmin()** and **argmax()** methods

```
a = np.array([[1,6,5],  
[4,3,7]])
```

```
# minimum along a column
```

```
print('Min :',np.argmin(a,axis=0))
```

```
# maximum along a row
```

```
print('Max :',np.argmax(a,axis=1))
```

Output?

- Let me break down the output for you. The minimum value for the first column is the first element along the column. For the second column, it is the second element. And for the third column, it is the first element.
- You can similarly determine what the output for maximum values indicates.

Introduction to Pandas

Pandas is a fast, powerful, flexible and easy to use open source data analysis and manipulation tool

Among its scientific computation libraries, Pandas is one of the most useful for data science operations. Pandas, along with Scikit-learn provides almost the entire stack needed by a data scientist.

Installing with Anaconda

```
pip install pandas
```

Series

The first main data type we will learn about for pandas is the Series data type

A Series is very similar to a NumPy array (in fact it is built on top of the NumPy array object). What differentiates the NumPy array from a Series, is that a Series can have axis labels, meaning it can be indexed by a label, instead of just a number location. It also doesn't need to hold numeric data, it can hold any arbitrary Python Object

```
import numpy as np
```

```
import pandas as pd
```

Creating a Series

We can convert a list, numpy array, or dictionary to a Series

```
labels = ['a','b','c']
```

```
my_list = [10,20,30]
```

```
arr = np.array([10,20,30])
```

```
d = {'a':10,'b':20,'c':30}
```

```
pd.Series(data=my_list)
```

Output:

```
0    10
```

```
1    20
```

```
2    30
```

try this `pd.Series(data=my_list,index=labels)` ?

`pd.Series(my_list,labels) ?`

`pd.Series(arr) ?`

`pd.Series(arr,labels) ?`

Using an Index

- The key to using a Series is understanding its index. Pandas makes use of these index names or numbers by allowing for fast look ups of information (works like a hash table or dictionary).

```
ser1 = pd.Series([1,2,3,4],index = ['USA', 'Germany','USSR', 'Japan'])
```

Ser1

Output:

USA 1

Germany 2

USSR 3

Japan 4

DataFrames

- DataFrames are the workhorse of pandas and are directly inspired by the R programming language. We can think of a DataFrame as a bunch of Series objects put together to share the same index

```
import pandas as pd
```

```
import numpy as np
```

```
from numpy.random import randn
```

```
np.random.seed(101)
```

```
df = pd.DataFrame(randn(5,4), "A B C D E".split(), "W X Y Z".split())
```

Df

Output:

	W	X	Y	Z
A	2.706850	0.628133	0.907969	0.503826
B	0.651118	-0.319318	-0.848077	0.605965
C	-2.018168	0.740122	0.528813	-0.589001
D	0.188695	-0.758872	-0.933237	0.955057
E	0.190794	1.978757	2.605967	0.683509

Selection and Indexing

`df['W']`

Output:

A	2.706850
B	0.651118
C	-2.018168
D	0.188695
E	0.190794

Removing Columns

`df.drop('new',axis=1)`

Missing Data

methods to deal with Missing Data in pandas

```
import numpy as np
```

```
import pandas as pd
```

```
df = pd.DataFrame({'A':[1,2,np.nan],  
                  'B':[5,np.nan,np.nan],  
                  'C':[1,2,3]})
```

Df

	A	B	C
0	1.0	5.0	1
1	2.0	NaN	2
2	NaN	NaN	3

```
mean = df['A'].mean()
```

```
df['A'] = df['A'].fillna(value=mean)
```

Df

	A	B	C
0	1.0	5.0	1
1	2.0	NaN	2
2	1.5	NaN	3

Merging, Joining, and Concatenating

```
import pandas as pd
```

```
df1 = pd.DataFrame({'A': ['A0', 'A1', 'A2', 'A3'],  
                    'B': ['B0', 'B1', 'B2', 'B3'],  
                    'C': ['C0', 'C1', 'C2', 'C3'],  
                    'D': ['D0', 'D1', 'D2', 'D3']},  
                    index=[0, 1, 2, 3])
```

```
df2 = pd.DataFrame({'A': ['A4', 'A5', 'A6', 'A7'],  
                    'B': ['B4', 'B5', 'B6', 'B7'],  
                    'C': ['C4', 'C5', 'C6', 'C7'],  
                    'D': ['D4', 'D5', 'D6', 'D7']},  
                    index=[4, 5, 6, 7])
```

```
df3 = pd.DataFrame({'A': ['A8', 'A9', 'A10', 'A11'],  
                    'B': ['B8', 'B9', 'B10', 'B11'],  
                    'C': ['C8', 'C9', 'C10', 'C11'],  
                    'D': ['D8', 'D9', 'D10', 'D11']},  
                    index=[8, 9, 10, 11])
```

Concatenation basically glues together DataFrames. Keep in mind that dimensions should match along the axis you are concatenating on. You can use **pd.concat** and pass in a list of DataFrames to concatenate together

```
pd.concat([df1,df2,df3])
```

Output:

	A	B	C	D
0	A0	B0	C0	D0
1	A1	B1	C1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3
4	A4	B4	C4	D4
5	A5	B5	C5	D5
6	A6	B6	C6	D6
7	A7	B7	C7	D7
8	A8	B8	C8	D8
9	A9	B9	C9	D9
10	A10	B10	C10	D10
11	A11	B11	C11	D11

The **merge** function allows you to merge DataFrames together using a similar logic as merging SQL Tables together

Joining is a convenient method for combining the columns of two potentially differently-indexed DataFrames into a single result DataFrame

There are lots of operations with pandas that will be really useful to you, but don't fall into any distinct category

```
import pandas as pd
```

```
df =
```

```
pd.DataFrame({'col1':[1,2,3,4],'col2':[444,555,666,444],'col3':['abc','def','ghi','xyz']})
```

```
df.head()
```

Output:

	col1	col2	col3
0	1	444	abc
1	2	555	def
2	3	666	ghi
3	4	444	xyz

Get column and index names

`df.columns`

Output:

`Index(['col2', 'col3'])`

`df.index`

Output:

`RangeIndex(start=0, stop=4, step=1)`

Find Null Values or Check for Null Values

`df.isnull()`

Output:

	col2	col3
0	False	False
1	False	False
2	False	False
3	False	False

```
# Drop rows with NaN Values
```

```
df.dropna()
```

Output:

	col2	col3
0	444	abc
1	555	def
2	666	ghi
3	444	xyz

- **Data Input and Output**

How to read dataset in to pandas:

pandas can read a variety of file types using its `pd.read_` methods

```
import numpy as np
```

```
import pandas as pd
```

- **CSV Input**

```
df = pd.read_csv('example')
```

Df

- **Excel**

- Pandas can read and write excel files, keep in mind, this only imports data

```
pd.read_excel('Excel_Sample.xlsx',sheetname='Sheet1')
```

Data Visualization

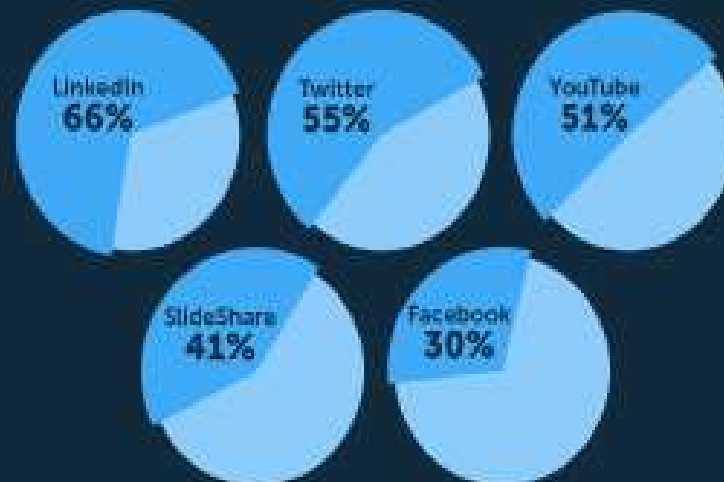
Traditional Vs. Visualization

TRADITIONAL STATISTICS

66% of B2B marketers rank LinkedIn as the most effective social media platform for their business. Other effective platforms include Twitter (55%), followed by YouTube (51%), SlideShare (41%) and Facebook with the smallest percent of marketer's selecting the platform for the most effective platform. (30%)

VISUALIZATION STATISTICS

B2B marketer's first choice for most effective social media platforms.



- Data visualization is the most important step in the life cycle of data science, data analytics, or we can say in data engineering.
- It is more impressive, interesting and understanding when we represent our study or analysis with the help of colors and graphics.
- Using visualization elements like graphs, charts, maps, etc., it becomes easier for clients to understand the underlying structure, trends, patterns and relationships among variables within the dataset.
- Simply explaining the data summary and analysis using plain numbers becomes complicated for both, people coming from technical and non-technical backgrounds.
- Data visualization gives us a clear idea of what the data wants to convey to us. It makes data neutral for us to understand the data insights.

- Data visualization involves operating a huge amount of data and converts it into meaningful and knowledgeable visuals using various tools.
- For visualizing data we need the best software tools to handle various types of data in structured or unstructured format from different sources such as files, web API, databases, and many more.
- We must choose the best visualization tool that fulfils all our requirements.
- The tool should support interactive plots generation, connectivity to data sources, combining data sources, automatically refresh the data, secured access to data sources, and exporting widgets.
- All these features allow us to make the best visuals of our data and also save time

Advantages of Data Visualization:



[Source : AV](#)

- **Easy to understand:** Managers and decision-makers use data visualization tools to create plots easily and rapidly consume important metrics.
- These metrics show the clear cut growth or loss in business. For example, if Sales are significantly going down in one region, decision-makers will easily find out from the data what circumstances or decisions are at present and how to respond to the factors encountered.
- Through graphical representations, we can interpret the vast features of data very clearly and cohesively, which allows us to understand the data and to draw conclusions from those insights and see business outlook.

Quick Decision Making: The Human mind can process visual images faster than texts and numerical values.

- Hence, seeing a graph, chart, or other visual and graphical representations of data is more pleasant and easy for our brain to process.
- To read and graph text, and then convert this into a visualization of the data that might not be entirely accurate becomes difficult and time consuming to understand for the team of decision-makers.
- It is a good human ability that easily interprets visual data; data visualization completely proves to improve the speed of decision-making processes. Data visualization always helps to shorten business meetings and efficient decision making.

- **Better Analysis:** Data visualization plays an important role for business stakeholders to analyze reports of business regarding sales, marketing strategies, and product interest.
- Better analysis can put our focus on the areas that require more attention to improve the strategies that increase profits and make the business more productive.
- **Identifying patterns:** Huge amount of sophisticated data will give several opportunities for insights after we visualize them.
- Visualization permits business users to acknowledge relationships and patterns between the data, also providing bigger meaning to it.
- Exploring these patterns helps users concentrate on specific areas that need attention within the data, to establish the importance of these areas to drive their business forward.

- **Detecting Errors:** Visualizing our data helps quickly determine any errors within the data.
- If the data tend to counsel the incorrect actions, visualization facilitates detecting inaccurate data sooner so it will be off from the analysis.
- **Exploring business insights:** Within the current competitive business atmosphere, finding data correlations using visual representations is essential to characterize business insights.
- Exploring these insights is very important for business users or executives to line up the correct path to achieving the business goals.
- **Efficient Storytelling:** Data visualizations are acknowledged because of the method of displaying data to produce insights that will support better decisions i.e., telling the story behind the data. It can offer factors, raise and draw attention to crucial insights and visually beat the other's business.

- **Discovery of Latest Trends within the Market:** Using data visualization, we'll be able to discover the most recent trends in your business to produce a quality product and determine issues before they arise.
- Staying on high of trends, we'll be able to place a lot of effort into augmented profits for our business.

Data Visualization with Pandas:

- Pandas library in python is mainly used for data analysis.
- It is not a data visualization library but, we can create basic plots using Pandas.
- Pandas is highly useful and practical if we want to create exploratory data analysis plots.
- As Pandas is Python's popular data analysis library, it provides several different functions to visualizing our data with the help of the `.plot()` function.
- There is one more advantage of using Pandas for visualization is we can serialize or create a pipeline of data analysis functions and plotting functions. It simplifies the task

Libraries for Data Visualization in Python

As they said a picture is better than a thousand words, executives and non-tech decision-makers heavily rely on and prefer visualizations and infographics to understand the underlying business insights and convey/pitch to the stakeholder.

Data-driven decision making is making its way to the top and we need to make sure to stay upbeat with the tools for the art.

Matplotlib:

- The Python library, to create stories with data. Another library from the SciPy Stack, Matplotlib plots 2D figures mainly.

When to use?

Matplotlib is the quintessential plotting library for Python that provides an object-oriented API for embedding plots into project files or applications.

What can you do with Matplotlib?

- Histogram,
- Bar plots,
- Scatter plots
- Area plot
- Pie charts,

Matplotlib can depict a wide range of 2D visualizations.

Matplotlib also facilitates labels, grids, legends, and some more formatting entities. Basically, everything that can be drawn

Seaborn

- it is defined as the data visualization library based on Matplotlib that provides a high-level interface for drawing attractive and informative statistical graphics.
- Putting it simply, *seaborn* is an extension of Matplotlib with advanced features.
- what is the difference between Matplotlib and Seaborn?

Matplotlib is used for basic plotting; bars, pies, lines, scatter plots and stuff whereas, *seaborn* provides a variety of advanced visualization patterns with less complex and fewer syntax.

What can you do with Seaborn?

1. Determine relationships between multiple variables (correlation)
2. Observe categorical variables for aggregate statistics
3. Analyze uni-variate or bi-variate distributions and compare them between different data subsets
4. Plot linear regression models for dependent variables
5. Provide high-level abstractions, multi-plot grids

Plotly

- Plotly is again a graph plotting library for Python. Users can import, copy, paste, or stream data that is to be analyzed and visualized.
- Each data viz library in Python varies from one another while also overlaps on some features. Plotly has many overlaps with *matplotlib* in terms of the range of viz available and *seaborn* for the advanced features.

When to use Plotly?

You can use Plotly if you want to create and display figures, update figures, hover over text for details. Plotly also has an additional feature of sending data to cloud servers. That's interesting!

What can you do with Plotly?

- The Plotly graph library has a wide range of graphs that you can plot:

1. *Basic Charts:* Line, Pie, Scatter, Bubble, Dot, Gantt, Sunburst, Treemap, Filled Area Charts

2. *Statistical and Seaborn Styles:* Error, Box, Histograms, Facet and Trellis Plots, Tree plots, Violin Plots, Trend Lines

3. *Scientific charts:* Contour, Ternary, Log, Quiver, Carpet, Radar, Heat maps, Windrose and Polar Plots

4. Financial Charts

5. Maps

6. Subplots

7. Transforms

8. Jupyter Widgets Interaction

Matplotlib

- Installation:

```
pip install matplotlib
```

- Import:

To import Matplotlib in Python, the most common and recommended way is to import its pyplot module and assign it the alias plt. This is done as follows:

```
import matplotlib.pyplot as plt
```

- Checking Matplotlib Version:

The version string is stored under `__version__` attribute.

```
import matplotlib.pyplot as plt
```

```
print(matplotlib.__version__)
```

Simple Line Plot

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Generate data
```

```
x = np.linspace(0, 10, 100)
```

```
y = np.sin(x)
```

```
# Plot
```

```
plt.plot(x, y)
```

```
plt.xlabel("x")
```

```
plt.ylabel("sin(x)")
```

```
plt.title("Simple Line Plot")
```

```
plt.grid(True)
```

```
plt.show()
```

Simple Scatter Plot

```
import numpy as np
import matplotlib.pyplot as plt

# Generate random scatter data
N = 200
x_sc = np.random.randn(N)
y_sc = np.random.randn(N)

# Plot
plt.scatter(x_sc, y_sc, alpha=0.5)
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Simple Scatter Plot")
plt.show()
```

Histogram

```
import numpy as np
import matplotlib.pyplot as plt
marks = [90,50,40,60,55,44,30,10,34,84]
grade_intervals = [0,35,70,100]
plt.title("STUDENTS GRADE")
plt.hist(marks,grade_intervals,histtype = "stepfilled")
plt.xticks([0,35,70,100])
plt.xlabel("Percentage")
plt.ylabel("No. of Students")
plt.show()
```

Histogram (Binning)

```
import numpy as np
import matplotlib.pyplot as plt
marks = [90,50,40,60,55,44,30,10,34,84]
plt.title("STUDENTS GRADE")
plt.hist(marks,bins=10,histtype = "stepfilled")
plt.xlabel("Percentage")
plt.ylabel("No. of Students")
plt.show()
```

Histogram (Density)

```
import numpy as np
import matplotlib.pyplot as plt
marks = [90,50,40,60,55,44,30,10,34,84]
plt.title("STUDENTS GRADE")
plt.hist(marks,bins=10,density = True, histtype = "stepfilled")
plt.xlabel("Percentage")
plt.ylabel("Density of Students")
plt.show()
```